# Unidad 3: Preprocesamiento y análisis morfosintáctico con spaCy

Cuando entregues tu cuaderno, debes nombrar el fichero (.ipynb) de la siguiente manera:

(3 últimos dígitos del alumno 1)_(3 últimos dígitos del alumno 2)_(UD+número)

Por ejemplo:

- Dos alumnos entregan las actividades evaluables de la Unidad 3
- Identificador alumno 1: 200800331
- Identificador alumno 2: 200800229

Nombre del fichero: `331_229_UD3.ipynb`

## 🚫 AVISO IMPORTANTE: PROHIBIDO EL USO DE IA

**Está prohibido usar cualquier herramienta de inteligencia artificial** (ChatGPT, Gemini, Copilot, la ayuda de IA de Google Colab, etc.) **para resolver las actividades**. La única excepción son las partes de una actividad en las que se pida explícitamente su uso: en ese caso, solo puedes usarla en ese momento concreto y únicamente la herramienta indicada.

**Las interpretaciones de resultados y las preguntas de reflexión deben resolverse siempre a partir de tu propio trabajo intelectual, sin ningún apoyo de una IA.**

## ⚠️ Normas generales de la actividad

- 🧑‍🏫 **El código debe ser el enseñado en el aula.** Aunque muchas actividades pueden resolverse de distintas formas (hay muchas maneras de llegar a un resultado solicitado), el código que emplees para resolver cada una de las actividades solicitadas tiene que ser el mismo que hemos aprendido en clase.
- ✍️ **Las preguntas de reflexión** debes responderlas **con tus propias palabras, sin ayuda alguna**.
- 📎 **Textos de la actividad:** los textos con los que tienes que trabajar no se reproducen en este documento: los encontrarás en ficheros de texto plano ( `.txt` ) en el paquete de materiales de la unidad. Ábrelos, copia el texto y pégalo en el código bajo la variable indicada en cada actividad. Ficheros: `texto_cliente.txt` .
- 📄 **Cuaderno de entrega:** trabaja sobre la plantilla `Plantilla_Evaluable_PLN3.ipynb` (contiene solo los títulos de cada actividad y una celda vacía del tipo que necesitas: de código y/o de texto). Respeta esa estructura al entregar.

# 🧾 Actividad propuesta

## 🧭 Tarea simulada profesional:

Tu cliente farmacéutico te ha enviado un conjunto de fragmentos de prospectos de medicamentos. Debes aplicar el pipeline de limpieza y generar una versión procesada de los textos para integrarlos en el sistema de traducción automática que están desarrollando.

Guarda en una variable `texto_cliente` el texto que encontrarás en el material de trabajo `texto_cliente.txt` (copia el texto y pégalo en el código bajo dicha variable).

El archivo original puede estar plagado de estructuras informales, puntuación variable, palabras técnicas y duplicidades. Tu misión es limpiar y lematizar todo el corpus.

Al final, entrega:

- Script con cada paso explicado (con #): normalización (conversión a minúsculas), eliminación de signos de puntuación, tokenización, eliminación de stopwods, lematización de tokens filtrados, glosario preliminar con sus frecuencias ordenado alfabéticamente y glosario técnico (sustantivos) con su contexto. Imprímelo todo por pantalla.
- **Resumen de tu pipeline (120-150 palabras)**: en una celda de texto (markdown), escribe un párrafo en prosa (no una lista) en el que cuentes al cliente farmacéutico qué has hecho con su texto y por qué. Para cada paso de tu script (minúsculas, eliminación de puntuación, tokenización, eliminación de stopwords, lematización y extracción de sustantivos), indica en una frase:
    - **qué hiciste** (por ejemplo: «se pasó todo el texto a minúsculas»);
    - **por qué lo hiciste**: qué problema del texto resuelve o qué aporta a un sistema de traducción automática (por ejemplo: «para que *Dosis* y *dosis* se cuenten como la misma palabra»).

    Termina con una frase que describa el resultado final: qué tipo de texto o glosario obtiene el cliente y para qué le sirve.

    💡 No se trata de pegar el código ni de repetir los comentarios `#` del script, sino de **justificar** tus elecciones con argumentos lingüísticos, como se lo explicarías a alguien que no programa.

**Objetivo:** Aplicar el pipeline de procesamiento de lenguaje natural aprendido para limpiar, tokenizar y lematizar textos técnicos reales.

| Criterio | ¿Qué se espera de ti? |
|---|---|
| **Pipeline bien estructurado** | Has aplicado los pasos del preprocesamiento en el orden correcto: normalización, limpieza, tokenización, eliminación de stopwords y lematización. |
| **Uso correcto de spaCy** | Has cargado el modelo español, procesado el texto con `nlp()` y usado correctamente los objetos `doc`, `token`, etc. |
| **Limpieza y lematización adecuadas** | Has eliminado correctamente la puntuación y las palabras vacías, y presentado una lista clara de lemas relevantes. |

| Criterio | ¿Qué se espera de ti? |
|---|---|
| **Explicación paso a paso clara** | Cada paso de tu script está comentado, y explicas para qué sirve y qué transforma. |
| **Resumen lingüístico argumentado** | Has escrito un párrafo de 120-150 palabras que recorre los pasos del pipeline en orden y justifica cada uno (qué hiciste y para qué sirve en un texto farmacéutico destinado a traducción automática), y cierras con una frase sobre el resultado obtenido. |

## 💬 Preguntas de reflexión

### Responde a las siguientes preguntas en una celda de markdown:

1. ¿Qué decisiones tomaste sobre qué palabras eliminar o conservar?
2. ¿Crees que en contextos técnicos deberíamos redefinir las “stopwords”?
3. ¿Qué pasaría si quitáramos palabras como “no”, “sin”, “antes”? ¿Tiene impacto?
4. ¿Detectaste palabras con múltiples significados que podrían afectar al entrenamiento?
5. ¿Cuáles crees que son las diferencias entre lematizar y simplemente dividir en palabras?

Objetivo: Demostrar comprensión crítica del impacto del preprocesamiento en el procesamiento del lenguaje natural técnico.

| Criterio | ¿Qué se espera de ti? |
|---|---|
| **Reflexión sobre stopwords** | Reconoces que en textos técnicos algunas palabras que suelen eliminarse (como "no" o "sin") pueden ser relevantes. |
| **Decisiones lingüísticas conscientes** | Explicas qué criterios usaste para conservar o eliminar términos y cómo podrían afectar al sistema de traducción. |
| **Conciencia de ambigüedades léxicas** | Detectas palabras que podrían tener varios significados y mencionas posibles riesgos en su interpretación automática. |
| **Diferencias entre lematizar y tokenizar** | Comparas adecuadamente ambos procesos y entiendes la ventaja de lematizar para tareas como traducción o análisis. |
| **Argumentación clara y personal** | Tus respuestas son personales, justificadas y bien conectadas con lo aprendido en la unidad. |

## 🧮 Rúbrica de evaluación

⚖️ **Peso de esta actividad en la nota final:** desarrollo del código **45 %** · resumen de decisiones **15 %** · preguntas de reflexión **40 %**

🧭 **Cómo se evalúa:** se valora **el procedimiento, no solo el resultado**. Resuelve la actividad con el método trabajado en el cuaderno principal de la unidad. Organizar el código a tu manera, usar otros nombres de variables o resolverlo de forma más breve **no penaliza**, siempre que se reconozca ese método. Lo que sí penaliza es sustituirlo por librerías o herramientas que no hemos trabajado y que no explicas.

| Criterio de evaluación | Excelente (9-10) | Notable (7-8) | Aprobado (5-6) | Insuficiente (0-4) |
|---|---|---|---|---|
| **Diseño del pipeline de limpieza y lematización** | Todos los pasos del preprocesamiento están presentes, en orden lógico y bien implementados. | El pipeline es claro y mayormente completo, con leves ausencias o errores. | El pipeline incluye los pasos esenciales, pero es poco claro o inconsistente. | El pipeline está incompleto, mal organizado o presenta errores graves. |
| **Uso de spaCy y herramientas NLP** | Uso correcto y eficiente de `spaCy` (modelo, tokens, lemas, PoS), bien integrado en el pipeline. | Uso adecuado de spaCy, aunque con aplicaciones algo básicas o repetitivas. | Uso mínimo o parcial de spaCy; hay funciones mal aplicadas. | No se usa spaCy correctamente o está ausente. |
| **Explicaciones y documentación del código** | Cada paso del código está bien comentado y explicado con claridad para el lector. | Las explicaciones son correctas aunque no siempre detalladas. | Comentarios escasos o poco informativos. | Código sin comentarios o explicaciones confusas. |
| **Reflexión lingüística y técnica** | Argumentación sólida sobre las decisiones de limpieza, stopwords y contexto técnico. | Reflexión clara pero algo superficial. | Reflexión limitada o genérica. | Reflexión incoherente o ausente. |
| **Presentación del trabajo y organización general** | Trabajo bien presentado, ordenado, con estructura clara y sin errores formales. | Presentación adecuada con leves problemas de formato o claridad. | Presentación irregular, aunque comprensible. | Presentación desordenada o difícil de seguir. |